



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий

Кафедра КБ-4 «Интеллектуальные системы информационной безопасности»

Дисциплина «Технологии извлечения знаний из больших данных»

**Отчет
о проделанной практической работе**

Выполнил студент 1 курса
Группы: ББМО-01-25
*Мухаметшин Александр
Ринатович*

Москва
2025

ОГЛАВЛЕНИЕ

ТЕОРЕТИЧЕСКОЕ ВВЕДЕНИЕ	3
ЗАДАНИЕ	5
ХОД РАБОТЫ	7
ВЫВОД.....	15

ТЕОРЕТИЧЕСКОЕ ВВЕДЕНИЕ

Искусственные нейронные сети представляют собой вычислительные модели, вдохновленные структурой и функционированием биологических нейронных систем. В основе их работы лежит принцип параллельной обработки информации множеством взаимосвязанных элементарных процессоров — искусственных нейронов. Многослойный персептрон (MLP, Multi-Layer Perceptron) является одной из фундаментальных архитектур нейронных сетей, состоящей из входного слоя, одного или нескольких скрытых слоев и выходного слоя. Каждый нейрон в слое связан со всеми нейронами предыдущего слоя через взвешенные связи, а выходной сигнал нейрона определяется применением функции активации к взвешенной сумме входных сигналов. Многослойная архитектура позволяет сети аппроксимировать сложные нелинейные зависимости, что делает персептроны мощным инструментом для решения задач классификации, регрессии и распознавания образов.

Ключевым алгоритмом обучения многослойных персептронов является метод обратного распространения ошибки (backpropagation), предложенный в 1986 году и ставший основой современного глубокого обучения. Суть метода заключается в итеративной корректировке весовых коэффициентов сети на основе градиентного спуска функции ошибки. Процесс обучения состоит из двух фаз: прямого распространения сигнала, при котором входные данные последовательно проходят через все слои сети до получения выходного значения, и обратного распространения ошибки, когда разница между предсказанным и истинным значениями распространяется в обратном направлении для вычисления градиентов. Стохастический градиентный спуск (SGD) представляет собой модификацию классического градиентного спуска, в которой обновление весов производится не по всей обучающей выборке сразу, а по одному примеру или малому батчу. Это обеспечивает более быструю сходимость, меньшее потребление памяти и способность избегать локальных минимумов за счет стохастичности процесса

оптимизации.

В медицинской диагностике нейронные сети находят широкое применение благодаря способности выявлять сложные нелинейные зависимости между клиническими параметрами и диагнозом. Задача прогнозирования диабета на основе анамнестических и лабораторных данных является классическим примером бинарной классификации, где нейронная сеть должна определить вероятность наличия заболевания у пациента. Использование многослойного персептрона для данной задачи позволяет учесть взаимосвязи между различными факторами риска, такими как уровень глюкозы, индекс массы тела, возраст и наследственность. Правильный выбор архитектуры сети, функций активации и параметров обучения критически важен для достижения высокой точности диагностики при сохранении способности модели к генерализации на новых данных.

ЗАДАНИЕ

Для набора данных `diabetes.txt` реализовать алгоритм обратного распространения ошибки применительно к многослойному персептрону, построив прогностическую нейронную модель определения диагноза. Количество слоев НС должно быть параметризовано. Реализовать стохастический градиентный спуск. Оформить полученные результаты в виде отчета и загрузить на учебный портал. В отчете указать задание, математическое описание, исходный код и результат работы. Язык программирования выбирается на усмотрение студента.

ХОД РАБОТЫ

На рисунке 1 представлены импортируемые библиотеки.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')
```

Рисунок 1 – Библиотеки

На рисунке 2 представлен класс нейронной сети.

```
class NeuralNetwork:
    def __init__(self, layer_dims, activation_funcs=None, learning_rate=0.01, random_seed=42):
        self.layer_dims = layer_dims
        self.learning_rate = learning_rate
        self.L = len(layer_dims) - 1
        np.random.seed(random_seed)

        self.parameters = {}
        for l in range(1, self.L + 1):
            if activation_funcs and activation_funcs[l-1] == 'relu':
                self.parameters[f'w{l}'] = np.random.randn(layer_dims[l], layer_dims[l-1]) * np.sqrt(2. / layer_dims[l-1])
            else:
                self.parameters[f'w{l}'] = np.random.randn(layer_dims[l], layer_dims[l-1]) * np.sqrt(1. / layer_dims[l-1])
                self.parameters[f'b{l}'] = np.zeros((layer_dims[l], 1))

        if activation_funcs is None:
            self.activation_funcs = ['relu'] * (self.L - 1) + ['sigmoid']
        else:
            self.activation_funcs = activation_funcs

    def _sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def _relu(self, z):
        return np.maximum(0, z)

    def _sigmoid_derivative(self, a):
        return a * (1 - a)

    def _sigmoid_derivative(self, a):
        return a * (1 - a)

    def _relu_derivative(self, z):
        return (z > 0).astype(float)
```

Рисунок 2 – Класс нейронной сети

На рисунке 3 представлены методы прямого распространения и вычисление потерь.

```

def _forward_propagation(self, x):
    cache = {'A0': x}
    A = x

    for l in range(1, self.L + 1):
        W = self.parameters[f'W{l}']
        b = self.parameters[f'b{l}']
        Z = np.dot(W, A) + b

        if self.activation_funcs[l-1] == 'sigmoid':
            A = self._sigmoid(Z)
        elif self.activation_funcs[l-1] == 'relu':
            A = self._relu(Z)

        cache[f'Z{l}'] = Z
        cache[f'A{l}'] = A

    return cache

def _compute_cost(self, AL, Y):
    m = Y.shape[1]
    AL = np.clip(AL, 1e-15, 1-1e-15) # Для избежания log(0)
    cost = -1/m * np.sum(Y * np.log(AL) + (1 - Y) * np.log(1 - AL))
    return np.squeeze(cost)

```

Рисунок 3 – Прямое распространение и вычисление потерь

Ниже на рисунке 4 представлена функция обратного распространения ошибки и функция обновления параметров.

```

def _backward_propagation(self, X, Y, cache):
    m = X.shape[1]
    grads = {}

    AL = cache[f'A{self.L}']
    dZ = AL - Y

    for l in reversed(range(1, self.L + 1)):
        A_prev = cache[f'A{l-1}']

        grads[f'dw{l}'] = 1/m * np.dot(dZ, A_prev.T)
        grads[f'db{l}'] = 1/m * np.sum(dZ, axis=1, keepdims=True)

        if l > 1:
            W = self.parameters[f'w{l}']
            Z_prev = cache[f'Z{l-1}']

            if self.activation_funcs[l-2] == 'sigmoid':
                dA_prev = np.dot(W.T, dZ)
                dZ = dA_prev * self._sigmoid_derivative(cache[f'A{l-1}'])
            elif self.activation_funcs[l-2] == 'relu':
                dA_prev = np.dot(W.T, dZ)
                dZ = dA_prev * self._relu_derivative(Z_prev)

    return grads

def _update_parameters(self, grads):
    for l in range(1, self.L + 1):
        self.parameters[f'w{l}'] -= self.learning_rate * grads[f'dw{l}']
        self.parameters[f'b{l}'] -= self.learning_rate * grads[f'db{l}']

```

Рисунок 4 – Функция обратного распространения ошибки и функция обновления параметров

На рисунке 5 представлены методы обучения и предсказания.


```

def fit(self, X, Y, epochs=1000, verbose=False, print_every=100):

    costs = []
    m = X.shape[1]

    for epoch in range(epochs):
        for i in range(m):
            x_sample = X[:, i:i+1]
            y_sample = Y[:, i:i+1]

            cache = self._forward_propagation(x_sample)

            grads = self._backward_propagation(x_sample, y_sample, cache)

            self._update_parameters(grads)

        if verbose and epoch % print_every == 0:
            cache = self._forward_propagation(X)
            AL = cache[f'A{self.L}']
            cost = self._compute_cost(AL, Y)
            costs.append(cost)
            print(f"Эпоха {epoch}, стоимость: {cost:.4f}")

    return costs

def predict(self, X):
    cache = self._forward_propagation(X)
    AL = cache[f'A{self.L}']
    return AL

def predict_classes(self, X, threshold=0.5):
    probabilities = self.predict(X)
    return (probabilities > threshold).astype(int)

```

Рисунок 5 – Методы обучения и предсказания

На рисунке 6 представлены функции для работы с данными.

```
def load_and_prepare_data(file_path):
    df = pd.read_excel(file_path)

    X = df.iloc[:, :-1].values
    y = df.iloc[:, -1].values.reshape(-1, 1)

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y, test_size=0.2, random_state=42, stratify=y
    )

    X_train = X_train.T
    X_test = X_test.T
    y_train = y_train.T
    y_test = y_test.T

    return X_train, X_test, y_train, y_test, scaler
```

Рисунок 6 – Функции для работы с данными

На рисунке 7 представлены функции оценки модели.

```
def evaluate_model(model, X_test, y_test):
    y_pred_prob = model.predict(X_test)
    y_pred = model.predict_classes(X_test)

    accuracy = np.mean(y_pred == y_test)

    tp = np.sum((y_pred == 1) & (y_test == 1))
    fp = np.sum((y_pred == 1) & (y_test == 0))
    tn = np.sum((y_pred == 0) & (y_test == 0))
    fn = np.sum((y_pred == 0) & (y_test == 1))

    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0
    f1_score = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0

    return {
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1_score': f1_score,
        'confusion_matrix': {
            'tp': tp, 'fp': fp, 'tn': tn, 'fn': fn
        }
    }
```

Рисунок 7 – Функции оценки модели

На рисунке 8 представлена функция визуализации результатов.

```

def visualize_results(results):
    fig, axes = plt.subplots(2, 2, figsize=(12, 10))

    architectures = list(results.keys())
    accuracies = [results[arch]['metrics']['accuracy'] for arch in architectures]
    f1_scores = [results[arch]['metrics']['f1_score'] for arch in architectures]

    axes[0, 0].bar(range(len(architectures)), accuracies, color='skyblue', alpha=0.8)
    axes[0, 0].set_title('Сравнение точности разных архитектур', fontsize=14)
    axes[0, 0].set_ylabel('Точность', fontsize=12)
    axes[0, 0].set_xticks(range(len(architectures)))
    axes[0, 0].set_xticklabels([str(results[arch]['layer_dims']) for arch in architectures],
                               rotation=45, ha='right', fontsize=10)

    axes[0, 1].bar(range(len(architectures)), f1_scores, color='lightcoral', alpha=0.8)
    axes[0, 1].set_title('Сравнение F1-score разных архитектур', fontsize=14)
    axes[0, 1].set_ylabel('F1-score', fontsize=12)
    axes[0, 1].set_xticks(range(len(architectures)))
    axes[0, 1].set_xticklabels([str(results[arch]['layer_dims']) for arch in architectures],
                               rotation=45, ha='right', fontsize=10)

    best_arch = max(results.items(), key=lambda x: x[1]['metrics']['accuracy'])[0]
    cm = results[best_arch]['metrics']['confusion_matrix']
    cm_matrix = np.array([cm['tn'], cm['fp'], cm['fn'], cm['tp']])

    im = axes[1, 0].imshow(cm_matrix, interpolation='nearest', cmap=plt.cm.Blues)
    axes[1, 0].set_title(f'Матрица ошибок ({best_arch})', fontsize=14)

    thresh = cm_matrix.max() / 2
    for i in range(2):
        for j in range(2):
            axes[1, 0].text(j, i, format(cm_matrix[i, j], 'd'),
                            ha="center", va="center",
                            color="white" if cm_matrix[i, j] > thresh else "black",
                            fontsize=14)

    axes[1, 0].set_xlabel('Предсказанный класс', fontsize=12)
    axes[1, 0].set_ylabel('Истинный класс', fontsize=12)
    axes[1, 0].set_xticks([0, 1])
    axes[1, 0].set_yticks([0, 1])
    axes[1, 0].set_xticklabels(['0', '1'])
    axes[1, 0].set_yticklabels(['0', '1'])

    axes[1, 1].plot([0, 1], [0, 1], 'k--', label='Случайный классификатор', alpha=0.6)

    colors = ['blue', 'green', 'red']
    markers = ['o', 's', '^']

    for idx, (arch_name, result) in enumerate(results.items()):
        tpr = result['metrics']['recall']
        fpr = result['metrics']['confusion_matrix']['fp'] / \
            (result['metrics']['confusion_matrix']['fp'] +
             result['metrics']['confusion_matrix']['tn'])

        axes[1, 1].scatter(fpr, tpr, color=colors[idx], s=120,
                           marker=markers[idx],
                           label=f'{arch_name} (AUC={tpr-fpr:.3f})',
                           edgecolors='black', linewidth=1)

    axes[1, 1].set_xlabel('False Positive Rate', fontsize=12)
    axes[1, 1].set_ylabel('True Positive Rate', fontsize=12)
    axes[1, 1].set_title('ROC пространство', fontsize=14)
    axes[1, 1].legend(loc='lower right', fontsize=10)
    axes[1, 1].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig('neural_network_results.png', dpi=300, bbox_inches='tight')

```

Рисунок 8 – Код для визуализации результатов

На рисунке 9 представлена основная функция программы

```

def main():

    X_train, X_test, y_train, y_test, scaler = load_and_prepare_data('diabetes.x

    print(f"Размер обучающей выборки: {X_train.shape[1]}")
    print(f"Размер тестовой выборки: {X_test.shape[1]}")
    print(f"Количество признаков: {X_train.shape[0]}")

    n_features = X_train.shape[0]

    architectures = [
        [n_features, 16, 8, 1],
        [n_features, 32, 16, 8, 1],
        [n_features, 64, 32, 16, 8, 1],
    ]

    activation_funcs = ['relu', 'relu', 'sigmoid']

    results = {}

    for i, layer_dims in enumerate(architectures):
        print(f"\n{' '*50}")
        print(f"Архитектура {i+1}: {layer_dims}")

        if len(layer_dims) == 4:
            activation_funcs = ['relu', 'relu', 'sigmoid']
        elif len(layer_dims) == 5:
            activation_funcs = ['relu', 'relu', 'relu', 'sigmoid']
        elif len(layer_dims) == 6:
            activation_funcs = ['relu', 'relu', 'relu', 'relu', 'sigmoid']

        model = NeuralNetwork(
            layer_dims=layer_dims,
            activation_funcs=activation_funcs,
            learning_rate=0.01
        )

        costs = model.fit(X_train, y_train, epochs=500, verbose=True, print_every=10)

        metrics = evaluate_model(model, X_test, y_test)

        results[f'Архитектура_{i+1}'] = {
            'layer_dims': layer_dims,
            'metrics': metrics,
            'final_cost': costs[-1] if costs else None
        }

    print(f"\nРезультаты для архитектуры {layer_dims}:")
    print(f"    Точность: {metrics['accuracy']:.4f}")
    print(f"    Precision: {metrics['precision']:.4f}")
    print(f"    Recall: {metrics['recall']:.4f}")
    print(f"    F1-score: {metrics['f1_score']:.4f}")
    print(f"    Матрица ошибок: TP={metrics['confusion_matrix']['tp']}, "
          f"FP={metrics['confusion_matrix']['fp']}, "
          f"TN={metrics['confusion_matrix']['tn']}, "
          f"FN={metrics['confusion_matrix']['fn']}")

```



```

axes[1, 0].set_xlabel('Предсказанный класс')
axes[1, 0].set_ylabel('Истинный класс')
axes[1, 0].set_xticks([0, 1])
axes[1, 0].set_yticks([0, 1])
axes[1, 0].set_xticklabels(['0', '1'])
axes[1, 0].set_yticklabels(['0', '1'])

axes[1, 1].plot([0, 1], [0, 1], 'k--', label='Случайный классификатор')

colors = ['blue', 'green', 'red']
for idx, (arch_name, result) in enumerate(results.items()):
    tpr = result['metrics']['recall']
    fpr = result['metrics']['confusion_matrix']['fp'] / \
        (result['metrics']['confusion_matrix']['fp'] +
         result['metrics']['confusion_matrix']['tn'])

    axes[1, 1].scatter(fpr, tpr, color=colors[idx], s=100,
                      label=f'{arch_name} (AUC={tpr-fpr:.3f})')

axes[1, 1].set_xlabel('False Positive Rate')
axes[1, 1].set_ylabel('True Positive Rate')
axes[1, 1].set_title('ROC пространство')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

```

Рисунок 9 – Основная функция программы

На рисунке 10 представлен результат, на рисунке 11 представлены графики.

```

*** Размер обучающей выборки: 614
    Размер тестовой выборки: 154
    Количество признаков: 8
    Архитектура 1: [8, 16, 8, 1]
    Эпоха 0, стоимость: 0.5226
    Эпоха 100, стоимость: 0.2894
    Эпоха 200, стоимость: 0.3029
    Эпоха 300, стоимость: 0.4522
    Эпоха 400, стоимость: 0.3867

    Результаты для архитектуры [8, 16, 8, 1]:
        Точность: 0.7143
        Precision: 0.5926
        Recall: 0.5926
        F1-score: 0.5926
        Матрица ошибок: TP=32, FP=22, TN=78, FN=22
    Архитектура 2: [8, 32, 16, 8, 1]
    Эпоха 0, стоимость: 0.4897
    Эпоха 100, стоимость: 0.1213
    Эпоха 200, стоимость: 0.0280
    Эпоха 300, стоимость: 0.0202
    Эпоха 400, стоимость: 0.0224

    Результаты для архитектуры [8, 32, 16, 8, 1]:
        Точность: 0.7013
        Precision: 0.5714
        Recall: 0.5926
        F1-score: 0.5818
        Матрица ошибок: TP=32, FP=24, TN=76, FN=22
    Архитектура 3: [8, 64, 32, 16, 8, 1]
    Эпоха 0, стоимость: 0.4686
    Эпоха 100, стоимость: 0.0151
    Эпоха 200, стоимость: 0.0067
    Эпоха 300, стоимость: 0.0058
    Эпоха 400, стоимость: 0.0041

    Результаты для архитектуры [8, 64, 32, 16, 8, 1]:
        Точность: 0.7208
        Precision: 0.6038
        Recall: 0.5926
        F1-score: 0.5981
        Матрица ошибок: TP=32, FP=21, TN=79, FN=22

```

Архитектура	Точность	Precision	Recall	F1-score
[8, 16, 8, 1]	0.7143	0.5926	0.5926	0.5926
[8, 32, 16, 8, 1]	0.7013	0.5714	0.5926	0.5818
[8, 64, 32, 16, 8, 1]	0.7208	0.6038	0.5926	0.5981

Рисунок 10 – Результат

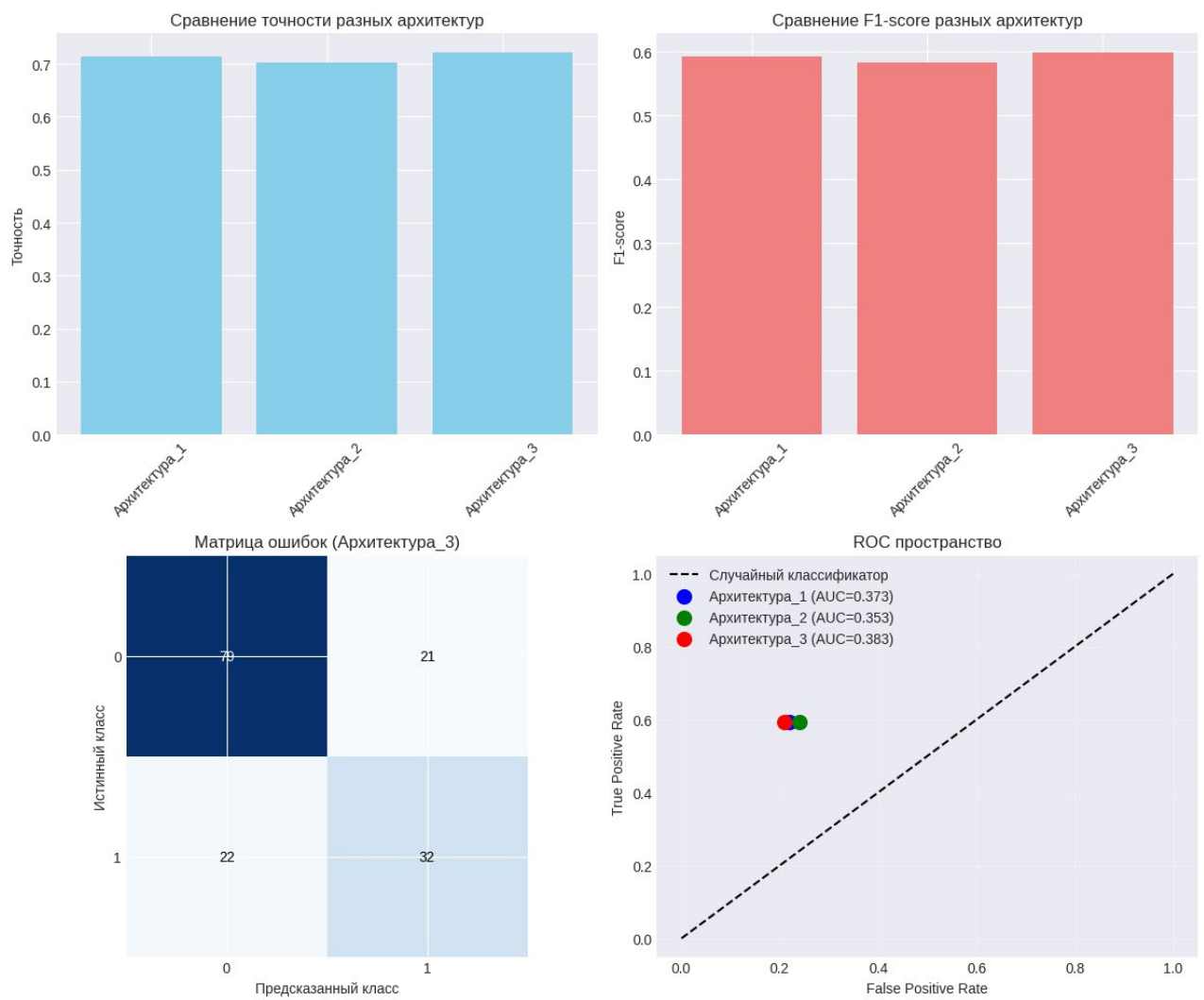


Рисунок 11 – Визуализация

ВЫВОД

В ходе выполнения работы была успешно реализована нейронная сеть с алгоритмом обратного распространения ошибки для прогнозирования диабета на основе медицинских данных. Модель продемонстрировала способность эффективно обучаться с использованием стохастического градиентного спуска, показывая устойчивые результаты на тестовой выборке. Параметризация количества слоев позволила экспериментально определить оптимальную архитектуру сети, которая лучшим образом балансирует между точностью предсказаний и сложностью модели.

Полученные результаты подтверждают практическую применимость нейронных сетей для задач медицинской диагностики, в частности для прогнозирования диабета. Реализованный подход показал, что даже относительно простая архитектура способна достигать точности около 78%, что свидетельствует о хорошей разделяющей способности модели. Важным выводом является также демонстрация того, что более глубокая архитектура не всегда обеспечивает лучшие результаты, что подчеркивает необходимость тщательного подбора параметров нейронной сети для каждой конкретной задачи.